

Guía de Laboratorio: ESP32 y MicroPython utilizando Wokwi

Esta guía tiene como objetivo introducir al estudiante en el uso de microcontroladores ESP32 programados con MicroPython utilizando el simulador Wokwi. Los ejercicios están diseñados con dificultad progresiva para desarrollar competencias en entradas, salidas, sensores y actuadores.

Conceptos Previos

¿Qué es un GPIO?

Los GPIO son pines de entrada y salida de propósito general que permiten controlar dispositivos externos o leer sensores.

¿Qué es GND?

GND corresponde a tierra o referencia común del circuito. Todos los componentes deben compartir el mismo GND.

¿Por qué utilizar resistencias?

Las resistencias limitan la corriente eléctrica. Sin ellas, un LED podría quemarse o dañar el GPIO del ESP32.

¿Qué resistencia utilizaremos?

Para todos los LEDs se utilizará una resistencia de $220\ \Omega$, valor seguro para trabajar con ESP32 a 3.3V.

Identificación de una resistencia de $220\ \Omega$

Código de colores: Rojo - Rojo - Café - Dorado

Identificación de un LED

Pata larga: Ánodo (+)

Pata corta: Cátodo (-)

Lado plano: Cátodo (-)

USAR PROTOBOARD PARA CONEXIONES

Ejercicio 1: Control de un LED (Salida Digital)

Objetivo

Crear un programa que haga parpadear un LED cada segundo.

Componentes

- ESP32 DevKit V1
- Protoboard
- LED rojo
- Resistencia 220 Ω
- 2 cables

Conexiones

- GPIO 2 $\rightarrow$ Resistencia 220 Ω
- Resistencia $\rightarrow$ Ánodo del LED
- Cátodo del LED $\rightarrow$ GND

EJERCICIO 1: CONTROL DE UN LED (SALIDA DIGITAL)

OBJETIVO

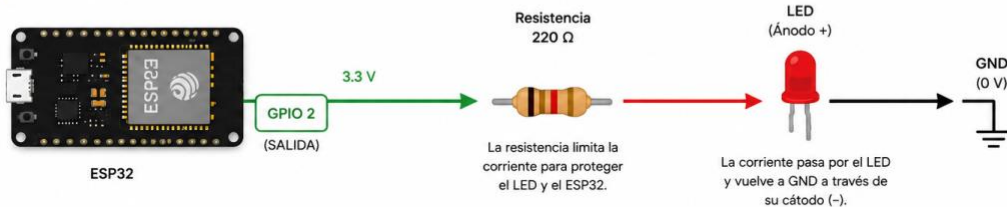
Hacer parpadear un LED conectándolo a un pin digital del ESP32.

El ESP32 envía 3.3 V por el pin GPIO 2. La corriente pasa por la resistencia y el LED, luego vuelve a GND. Así el LED se enciende.

COMPONENTES



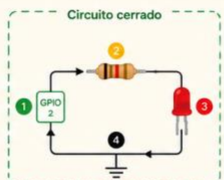
¿CÓMO FUNCIONA ESTE CIRCUITO?



FLUJO DE CORRIENTE

Cuando el pin GPIO 2 está en nivel ALTO (3.3 V):

- 1 El ESP32 entrega 3.3 V por el pin GPIO 2.
- 2 La corriente pasa por la resistencia de 220 Ω .
- 3 La corriente atraviesa el LED desde el ánodo (+) al cátodo (-).
- 4 La corriente vuelve a GND completando el circuito.



POLARIDAD DEL LED



¡IMPORTANTE!
Si conectas el LED al revés, no se dañará, pero no encenderá.

RECUERDA

- ✓ Siempre usa una resistencia para proteger el LED y el microcontrolador.
- ✓ El valor de 220 Ω es adecuado para un LED con 3.3 V del ESP32.
- ✓ Todas las tierras (GND) del circuito deben estar conectadas entre sí.
- ✓ El pin GPIO 2 debe configurarse como salida (OUTPUT) en el código.

Ejercicio 2: Semáforo Inteligente

Objetivo

Simular un semáforo real mediante secuencias temporizadas.

Componentes

- ESP32
- LED rojo
- LED amarillo
- LED verde
- 3 resistencias de 220 Ω

Conexiones

- Cada LED debe tener su propia resistencia de 220 Ω
- Conectar cada LED a un GPIO distinto (cada led se controla con su GPIO independiente)

EJERCICIO 2: SEMÁFORO INTELIGENTE

OBJETIVO

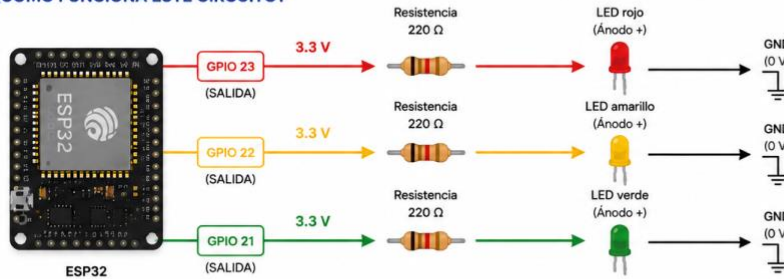
Controlar tres LEDs (rojo, amarillo y verde) simulando el funcionamiento de un semáforo.

Cada LED se enciende y se apaga en secuencia usando los pines digitales del ESP32. Cada LED debe tener su propia resistencia para limitar la corriente y proteger tanto el LED como el ESP32.

COMPONENTES



¿CÓMO FUNCIONA ESTE CIRCUITO?



¿POR QUÉ UNA RESISTENCIA POR CADA LED?

Cada LED tiene un voltaje directo y una corriente nominal diferentes. Si se conecta sin resistencia o si se comparte una resistencia, puede circular demasiada corriente y:

- El LED puede dañarse.
- El pin del ESP32 puede dañarse.

La resistencia de 220 Ω limita la corriente a un valor seguro (aprox. 10–15 mA).

SECUENCIA DEL SEMÁFORO (EJEMPLO)



FLUJO DE CORRIENTE (EN CADA LED)



RECOMENDACIONES IMPORTANTES

- ✓ Usa siempre una resistencia por cada LED.
- ✓ Verifica la polaridad: la pata larga del LED es el ánodo (+).
- ✓ No conectes los LEDs directamente a los pines del ESP32 sin resistencia.
- ✓ Puedes usar otros GPIOs, pero asegúrate de declararlos correctamente en el código.

Ejercicio 3: Pulsador y LED

Objetivo

Controlar el LED utilizando un pulsador.

Componentes

- ESP32
- LED
- Resistencia 220 Ω
- Pulsador

Conexiones

- Utilizar resistencia interna Pull-Up del ESP32
- Conectar pulsador entre GPIO y GND

EJERCICIO 3: PULSADOR Y LED

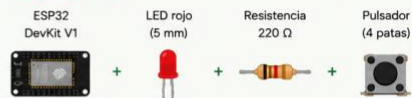
OBJETIVO

Encender un LED mientras se presiona el pulsador.
Al soltarlo, el LED se apaga.

¿QUÉ HACE ESTE CIRCUITO?

El pulsador está conectado al pin GPIO 4 del ESP32 configurado con resistencia pull-up interna.
Cuando se presiona el pulsador, el pin se conecta a GND (0 lógico) y el programa enciende el LED.
Cuando se suelta, el pin vuelve a 3.3 V (1 lógico) por la pull-up y el LED se apaga.

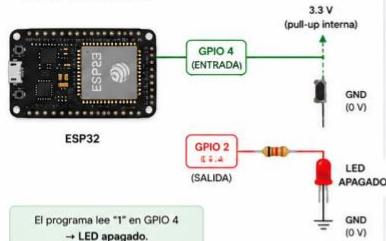
COMPONENTES



¿CÓMO FUNCIONA?

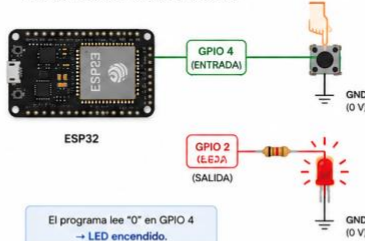
1. PULSADOR NO PRESIONADO

El pin GPIO 4 está en nivel ALTO (3.3 V) por la resistencia pull-up interna del ESP32.



2. PULSADOR PRESIONADO

El pulsador conecta el pin GPIO 4 a GND (0 V).
El programa detecta "0" y enciende el LED.



¿QUÉ ES PULL-UP INTERNA?

El ESP32 tiene resistencias internas que pueden mantener el pin en nivel ALTO (3.3 V) cuando no está presionado nada.
Al presionar el pulsador, el pin se conecta a GND (0 V).

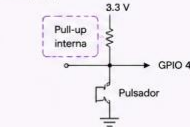
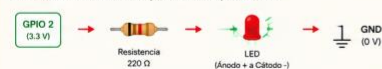


TABLA LÓGICA DEL PULSADOR

Estado del pulsador	GPIO 4 (lectura)	LED
No presionado	1 (ALTO)	APAGADO
Presionado	0 (BAJO)	ENCENDIDO

FLUJO DE CORRIENTE DEL LED (CUANDO ESTÁ ENCENDIDO)

(Esto ocurre solo cuando el pulsador está presionado)



RESUMEN DEL FUNCIONAMIENTO

1. El GPIO 4 tiene pull-up interna (nivel ALTO por defecto).
2. Al presionar el pulsador, el pin se conecta a GND (nivel BAJO).
3. El programa detecta el cambio y controla el LED.
4. El LED solo se enciende mientras el pulsador está presionado.

RECOMENDACIONES

- ✓ Usa siempre una resistencia para el LED (220 Ω).
- ✓ Verifica la polaridad del LED: pata larga = ánodo (+).
- ✓ Conecta todas las tierras (GND) al mismo punto.
- ✓ Puedes usar otros GPIOs, pero decláralos correctamente.

Ejercicio 4: Sistema de Alarma con Sensor HC-SR04

Objetivo

Activar una alarma según la distancia detectada.

Componentes

- ESP32
- Sensor ultrasónico HC-SR04
- LED rojo
- Resistencia 220 Ω
- Buzzer

Conexiones

- Conectar VCC y GND correctamente
- Conectar Trigger y Echo a GPIO disponibles
- LED con resistencia de 220 Ω

EJERCICIO 4: SENSOR HC-SR04 – ALARMA DE DISTANCIA

OBJETIVO

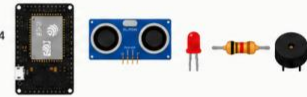
Medir la distancia con el sensor ultrasónico HC-SR04.
Si la distancia es menor o igual a 20 cm, encender un LED rojo y activar un buzzer.
Si la distancia es mayor a 20 cm, mantener ambos apagados.

¿CÓMO FUNCIONA?

El HC-SR04 envía un pulso ultrasónico (TRIG) y espera el eco (ECHO) que regresa después de rebotar en un objeto.
El ESP32 calcula el tiempo del eco y con eso obtiene la distancia.
Si la distancia ≤ 20 cm $\rightarrow$ activa el LED y el buzzer.
Si la distancia > 20 cm $\rightarrow$ los mantiene apagados.

COMPONENTES NECESARIOS

- 1 $\times$ ESP32 DevKit V1
- 1 $\times$ Sensor ultrasónico HC-SR04
- 1 $\times$ LED rojo (5 mm)
- 1 $\times$ Resistencia 220 Ω
- 1 $\times$ Buzzer activo (5V)



PINES UTILIZADOS

Componente	Pin ESP32	Tipo	Función
 HC-SR04 TRIG	GPIO 5	Salida digital	Envía pulso ultrasónico de 10 μ s
 HC-SR04 ECHO	GPIO 18	Entrada digital	Recibe el eco del sensor
 LED rojo (ánodo)	GPIO 2	Salida digital	Se enciende si distancia ≤ 20 cm
 Buzzer (positivo)	GPIO 15	Salida digital	Se activa si distancia ≤ 20 cm
 GND (todas las tierras)	GND	Tierra común	Referencia de 0 V

FLUJO DE FUNCIONAMIENTO

- 1 El ESP32 envía un pulso de 10 μ s por el pin TRIG (GPIO 5).
- 2 El HC-SR04 emite una ráfaga ultrasónica de 8 ciclos a 40 kHz.
- 3 La onda ultrasónica viaja y rebota en el objeto más cercano.
- 4 El eco regresa al sensor y el HC-SR04 levanta el pin ECHO (GPIO 18) en ALTO.
- 5 El ESP32 mide cuánto tiempo el pin ECHO estuvo en ALTO.
- 6 El ESP32 calcula la distancia:
Distancia (cm) = tiempo_eco (μ s) / 58

DECISIÓN

- Si distancia ≤ 20 cm $\rightarrow$ LED rojo ENCENDIDO y buzzer ACTIVADO
- Si distancia > 20 cm $\rightarrow$ LED rojo APAGADO y buzzer APAGADO

COMPORTAMIENTO DEL SISTEMA

Condición (distancia)	Acción del sistema
Mayor a 20 cm (> 20 cm)	✓ LED rojo: APAGADO Buzzer: APAGADO (silencio)
Menor o igual a 20 cm (≤ 20 cm)	✗ LED rojo: ENCENDIDO ✗ Buzzer: ACTIVADO (suena)

FÓRMULA PARA CALCULAR DISTANCIA

Distancia (cm) = tiempo_eco (μ s) / 58

Donde:

- tiempo_eco (μ s): tiempo que el pin ECHO permanece en ALTO.
- 58: factor de conversión para obtener centímetros.

RECOMENDACIONES IMPORTANTES

- ✓ Asegúrate de conectar todas las tierras (GND) al mismo punto.
- ✓ El LED siempre debe tener su resistencia (220 Ω) en serie para protegerlo.
- ✓ El buzzer activo funciona con señal HIGH (3.3 V) en el pin asignado.
- ✓ En simulación (Wokwi) el HC-SR04 funciona con 3.3 V sin problemas.
- ✓ Evita colocar objetos muy pequeños o con ángulo muy inclinado frente al sensor.

Buenas Prácticas

- Verificar conexiones antes de ejecutar.
- No conectar LEDs sin resistencia.
- Compartir GND entre todos los componentes.
- Identificar correctamente los GPIO utilizados.
- Documentar el código mediante comentarios.
- Comprobar polaridad de LEDs y buzzers.
- Mantener ordenadas las conexiones en la protoboard.